

APPENDIX

A. Proof of Theorem 1

We model a high-dimensional query as a probability vector on the $(K-1)$ -dimensional simplex:

$$\Delta^{K-1} = \left\{ \alpha \in \mathbb{R}^K : \alpha_i \geq 0, \sum_{i=1}^K \alpha_i = 1 \right\}. \quad (8)$$

Definition 2 (Dirichlet Distribution). *Let $\theta = (\theta_1, \dots, \theta_K)$ with $\theta_i > 0$. A random vector $\alpha \in \Delta^{K-1}$ follows a Dirichlet distribution, denoted by*

$$\alpha \sim \text{Dir}(\theta),$$

if its density is given by

$$\text{Dir}(\alpha; \theta) = \frac{1}{B(\theta)} \prod_{i=1}^K \alpha_i^{\theta_i-1}, \quad (9)$$

where

$$B(\theta) = \frac{\prod_{i=1}^K \Gamma(\theta_i)}{\Gamma\left(\sum_{i=1}^K \theta_i\right)}. \quad (10)$$

Definition 3 (Symmetric Dirichlet Distribution). *If $\theta_1 = \dots = \theta_K = \theta$, the distribution is called symmetric Dirichlet, denoted by*

$$\alpha \sim \text{Dir}(\theta; K).$$

Its density simplifies to

$$\text{Dir}(\alpha; \theta) = \frac{\Gamma(K\theta)}{\Gamma(\theta)^K} \prod_{i=1}^K \alpha_i^{\theta-1}. \quad (11)$$

Lemma 1 (Marginal Distribution). *Let $\alpha \sim \text{Dir}(\theta; K)$. Then each coordinate satisfies*

$$\alpha_i \sim \text{Beta}(\theta, (K-1)\theta), \quad (12)$$

and

$$\mathbb{E}[\alpha_i] = \frac{1}{K}. \quad (13)$$

Definition 4 (Top- m Mass). *Let $\alpha_{(1)} \geq \alpha_{(2)} \geq \dots \geq \alpha_{(K)}$ denote the ordered coordinates of α . Define the top- m mass as*

$$S_m = \sum_{i=1}^m \alpha_{(i)}.$$

Definition 5 (m -Sparse Dominance). *For $\varepsilon > 0$, we say α exhibits m -sparse dominance if*

$$S_m \geq 1 - \varepsilon.$$

Definition 6 (Threshold Count). *For $\tau > 0$, define*

$$C_\tau = \#\{i : \alpha_i > \tau\}.$$

Lemma 2. *For any $\varepsilon > 0$ and integer $m < K$,*

$$C_{\frac{\varepsilon}{K-m}} \leq m \implies S_m \geq 1 - \varepsilon.$$

Proof. If $C_{\frac{\varepsilon}{K-m}} \leq m$, then at most m coordinates exceed $\frac{\varepsilon}{K-m}$. Thus for all $i > m$,

$$\alpha_{(i)} \leq \frac{\varepsilon}{K-m}.$$

Hence,

$$\sum_{i=m+1}^K \alpha_{(i)} \leq (K-m) \cdot \frac{\varepsilon}{K-m} = \varepsilon.$$

Therefore,

$$S_m = 1 - \sum_{i=m+1}^K \alpha_{(i)} \geq 1 - \varepsilon. \quad \square$$

Lemma 3. *Let $\alpha \sim \text{Dir}(\theta; K)$. Then*

$$\mathbb{E}[C_\tau] = K \cdot \Pr(\alpha_i > \tau) = K(1 - I_\tau(\theta, (K-1)\theta)),$$

where $I_\tau(\cdot, \cdot)$ denotes the regularized incomplete Beta function.

Proof. By definition, C_τ counts the number of coordinates whose values exceed the threshold τ . Equivalently, it can be written as a sum of indicator variables:

$$C_\tau = \sum_{i=1}^K \mathbf{1}\{\alpha_i > \tau\}.$$

Taking expectation on both sides and using the linearity of expectation, we obtain

$$\mathbb{E}[C_\tau] = \sum_{i=1}^K \mathbb{E}[\mathbf{1}\{\alpha_i > \tau\}].$$

For each indicator variable,

$$\mathbb{E}[\mathbf{1}\{\alpha_i > \tau\}] = \Pr(\alpha_i > \tau).$$

Therefore,

$$\mathbb{E}[C_\tau] = \sum_{i=1}^K \Pr(\alpha_i > \tau).$$

Since $\alpha \sim \text{Dir}(\theta; K)$ is symmetric, all coordinates have the same marginal distribution. Hence,

$$\Pr(\alpha_1 > \tau) = \Pr(\alpha_2 > \tau) = \dots = \Pr(\alpha_K > \tau),$$

which gives

$$\mathbb{E}[C_\tau] = K \Pr(\alpha_i > \tau).$$

For a symmetric Dirichlet distribution, the marginal distribution of each coordinate is

$$\alpha_i \sim \text{Beta}(\theta, (K-1)\theta).$$

Thus, its cumulative distribution function is

$$\Pr(\alpha_i \leq \tau) = I_\tau(\theta, (K-1)\theta),$$

where $I_\tau(\cdot, \cdot)$ is the regularized incomplete Beta function:

$$I_x(a, b) = \frac{B_x(a, b)}{B(a, b)} = \frac{\int_0^x t^{a-1}(1-t)^{b-1} dt}{\int_0^1 t^{a-1}(1-t)^{b-1} dt}.$$

Here $B(a, b)$ denotes the two-parameter Beta function, which is the $K = 2$ special case of the multivariate Beta function $B(\theta)$. Therefore,

$$\Pr(\alpha_i > \tau) = 1 - I_\tau(\theta, (K-1)\theta).$$

Substituting this into the expression for $\mathbb{E}[C_\tau]$, we obtain

$$\mathbb{E}[C_\tau] = K(1 - I_\tau(\theta, (K-1)\theta)).$$

This completes the proof. $\square$

Theorem 4. Let $\alpha \sim \text{Dir}(\theta; K)$. Then for any $\varepsilon > 0$ and $m < K$,

$$\Pr(S_m \geq 1 - \varepsilon) \geq \Pr\left(C_{\frac{\varepsilon}{K-m}} \leq m\right).$$

Moreover,

$$\Pr(S_m \geq 1 - \varepsilon) \geq 1 - \frac{K}{m+1} \left(1 - I_{\frac{\varepsilon}{K-m}}(\theta, (K-1)\theta)\right).$$

Proof. The first inequality follows from the previous lemma.

For the second inequality, note that

$$C_\tau = \sum_{i=1}^K \mathbf{1}\{\alpha_i > \tau\}.$$

Thus,

$$\mathbb{E}[C_\tau] = K(1 - I_\tau(\theta, (K-1)\theta)).$$

Applying Markov's inequality,

$$\Pr(C_\tau \geq m+1) \leq \frac{\mathbb{E}[C_\tau]}{m+1}.$$

Therefore,

$$\Pr(C_\tau \leq m) \geq 1 - \frac{\mathbb{E}[C_\tau]}{m+1}.$$

Substituting $\tau = \frac{\varepsilon}{K-m}$ completes the proof. $\square$

Following [41], we set the parameter $\theta < 1$, specifically, $\theta = \frac{1}{K}$. When $\varepsilon = 0.1$, we can simulate the probability.

B. Proof of Theorem 2

Proof. We prove the two cases separately.

Case 1: $\alpha_i > 0$ for all i . Consider any object $v \notin \text{KSky}(\mathcal{D}, o, K)$. By the definition of K -skyband, v is dominated by at least K distinct objects. That is, there exist K objects u such that $f_i(u) \leq f_i(v)$ for all i , and for at least one coordinate the inequality is strict.

Since all $\alpha_i > 0$, multiplying each inequality by α_i and summing them preserves the strict inequality:

$$\sum_{i=1}^m \alpha_i f_i(u) < \sum_{i=1}^m \alpha_i f_i(v).$$

Thus, all these K objects have strictly smaller weighted distance than v . Hence v cannot rank within the top- κ for any $\kappa < K$. Therefore, every top- κ nearest neighbor must lie in $\text{KSky}(\mathcal{D}, o, K)$.

Case 2: Some $\alpha_i = 0$. Let T be any top- κ nearest-neighbor set of o under the weighted distance. If $T \subseteq \text{KSky}(\mathcal{D}, o, K)$, we are done.

Otherwise, choose an object $v \in T$ that is not in $\text{KSky}(\mathcal{D}, o, K)$. Then v is dominated by at least K objects. For each such dominator u , we have $f_i(u) \leq f_i(v)$ for all i . Since $\alpha_i \geq 0$, it follows that

$$\sum_{i=1}^m \alpha_i f_i(u) \leq \sum_{i=1}^m \alpha_i f_i(v).$$

Because $|T| = \kappa < K$, at least one dominator u is not in T . We may replace v in T with such a u without increasing the weighted distance. If the new element still lies outside $\text{KSky}(\mathcal{D}, o, K)$, we repeat the same replacement process. Since the dataset is finite, this process must terminate. Eventually we obtain a top- κ set entirely contained in $\text{KSky}(\mathcal{D}, o, K)$. $\square$

C. Proof of Theorem 3

Proof. We prove that every object in MLSky belongs to the K -skyband of CS .

Recall that L_i is defined as the skyline (i.e., 1-skyband) of the residual set $CS_i = CS \setminus \bigcup_{j=1}^{i-1} L_j$. By the definition of skyline, no object in CS_i dominates any object in L_i . Therefore, if an object $x \in L_i$ is dominated by some object in CS , such a dominator must belong to one of the earlier layers $\bigcup_{j=1}^{i-1} L_j$.

Now consider any object $x \in \text{MLSky}$. Then $x \in L_i$ for some $1 \leq i \leq \ell$. As argued above, all objects that dominate x must lie in $\bigcup_{j=1}^{i-1} L_j$. Hence, the number of objects in CS that dominate x is at most

$$\left| \bigcup_{j=1}^{i-1} L_j \right|.$$

Since $\sum_{j=1}^{\ell} |L_j| \leq K$, we have

$$\left| \bigcup_{j=1}^{i-1} L_j \right| = \sum_{j=1}^{i-1} |L_j| \leq \sum_{j=1}^{\ell-1} |L_j| \leq K-1.$$

Therefore, every object in MLSky is dominated by at most $K-1$ objects in CS . By the definition of the K -skyband, this implies that MLSky is a subset of the K -skyband of CS . $\square$

D. Time Complexity

The dominant cost of Algorithm 2 comes from SUBSPACE SKYLINE . For a candidate pool of size $n = |CS|$, it enumerates all $\binom{m}{2} = O(m^2)$ objective pairs (i, j) . For each pair, it sorts the candidates by $(f_i(v), f_j(v))$ in $O(n \log n)$ time and performs one linear scan in $O(n)$ time. Thus, the cost of computing one skyline layer is $O(m^2 n \log n)$. The peeling procedure extracts ℓ layers until the accumulated size reaches the budget K , where $\ell \leq \min(K, n)$. Therefore, the total time complexity is $O(\ell m^2 n \log n)$.

The overall computational cost of Algorithm 1 is primarily governed by the length of the search trajectory. LSS follows the standard beam-based neighbor-expansion paradigm (Lines 14-19) widely adopted in graph-based ANNS methods. In the extreme case where each call to SKYLINE LAYERS

Algorithm 4: Incremental Maintenance of Entry Points

Input: Current entry-point set EP ; current skyline set SK ; new object o ; center c ; size budget B

Output: Updated entry-point set EP , skyline set SK

```

1 Procedure UpdateEP( $EP, SK, o, c, B$ ):
2   Compute  $g(o)$  with  $g_i(o) = -d(o, \mathbf{e}_i, c, \mathbf{e}_i)$ 
3   if  $\exists v \in SK$  s.t.  $g(v) \succ g(o)$  then
4     return  $EP, SK$ 
5    $SK \leftarrow (SK \setminus \{v \in SK \mid g(o) \succ g(v)\}) \cup \{o\}$ 
6    $EP \leftarrow (EP \setminus \{v \in EP \mid g(o) \succ g(v)\}) \cup \{o\}$ 
7   if  $|EP| > B$  then
8      $U \leftarrow \emptyset; \quad R \leftarrow EP$ 
9     for  $i \leftarrow 1$  to  $m$  do
10       $T_i \leftarrow \lfloor B/m \rfloor$  smallest objects in  $R$  by  $g_i(\cdot)$ 
11       $U \leftarrow U \cup T_i; \quad R \leftarrow R \setminus T_i$ 
12     $EP \leftarrow U$ 
13 return  $EP, SK$ 

```

returns only a single candidate, LSS degenerates to a purely greedy expansion.

E. Index Construction

As shown in Algorithm 4, for a newly inserted object o , we compute $g(o)$, where $g(o) \succ g(v)$ denotes that $g(o)$ Pareto-dominates $g(v)$. If $g(o)$ is dominated by some $g(v)$ with $v \in SK$, o is discarded from entry-point consideration (Lines 2–4). Otherwise, we remove from SK and EP the objects dominated by $g(o)$, and insert o into both sets (Lines 5–6). If $|EP| > B$, we select $\lfloor B/m \rfloor$ objects with the smallest $g_i(\cdot)$ for each feature i , ensuring balanced coverage across features (Lines 7–12).

Algorithm 5 summarizes the full construction process. Starting from an initial object o_0 , we maintain the graph index G , entry points EP , and skyline set SK .

For each new object o : (1) we obtain a candidate set $CS(o)$ from EP using LSS (Algorithm 1); (2) we apply template-aware pruning (Algorithm 3) to select neighbors $NS(o)$; (3) we update reverse edges for affected neighbors; and (4) we update the entry-point set via Algorithm 4.

The reverse-edge re-pruning step (Line 12) compensates for the incremental insertion order. Since earlier nodes could not consider later insertions during their original neighbor selection, we re-apply the same pruning rule to $G[v] \cup \{o\}$ to ensure structural consistency.

F. WTNG Search Algorithm

Algorithm 6 presents the weight-aware beam search procedure on WTNG. Given a query q with weight vector α , the algorithm initializes a candidate min-heap $\mathcal{S}$ and a result max-heap $\mathcal{R}$ using the entry-point set EP . All entry points are evaluated by the aggregated distance $d_\alpha(q, \cdot)$ and inserted into both heaps.

Algorithm 5: Index Construction

Input: Dataset $\mathcal{D}$; candidate budget $ef_{\text{construction}}$; entry-point budget B ; maximum degree M ; template set $\mathcal{T}$

Output: Graph index G ; entry-point set EP

```

1 Procedure IndexBuild( $\mathcal{D}, ef_c, B, M, \mathcal{T}$ ):
2   Select the first object  $o_0 \in \mathcal{D}$  as the start node
3    $G[o_0] \leftarrow \emptyset$ 
4    $EP \leftarrow \{o_0\}, SK \leftarrow \{o_0\}$ 
5   Compute the dataset center  $c$ 
6   foreach  $o \in \mathcal{D} \setminus \{o_0\}$  do
7      $CS(o) \leftarrow \text{LSS}(G, o, EP, ef_{\text{construction}})$ 
8      $NS(o) \leftarrow \text{TemplatePrune}(o, CS(o), M, \mathcal{T})$ 
9      $G[o] \leftarrow NS(o)$ 
10    foreach  $v \in NS(o)$  do
11       $C_v \leftarrow G[v] \cup \{o\}$ 
12       $G[v] \leftarrow \text{TemplatePrune}(v, C_v, M, \mathcal{T})$ 
13     $EP, SK \leftarrow \text{UpdateEP}(EP, SK, o, c, B)$ 
14 return  $G, EP$ 

```

We then select the n_t templates in $\mathcal{T}$ that are closest to α , forming $\mathcal{T}_q$. During traversal, only edges whose bit masks are activated under at least one template in $\mathcal{T}_q$ are explored. This effectively induces a query-adaptive overlay graph.

The beam search proceeds by repeatedly extracting the closest candidate v from $\mathcal{S}$ and expanding its activated neighbors. The behavior differs depending on whether the result heap $\mathcal{R}$ has reached its capacity.

- $|\mathcal{R}| < ef_{\text{search}}$. The beam is not yet full. For each unvisited neighbor o of v , we compute its full aggregated distance $d_\alpha(q, o)$ and insert o into both $\mathcal{S}$ and $\mathcal{R}$. No pruning is applied in this phase.
- $|\mathcal{R}| = ef_{\text{search}}$. The result heap is full, and its farthest element defines the current threshold τ . If $d_\alpha(q, v) > \tau$, the search terminates since no better candidate can be found. Otherwise, neighbors are evaluated using the early-stopping routine (Algorithm 7), and only candidates with distance smaller than τ are inserted into the heaps.

The process continues until no candidate in $\mathcal{S}$ can improve the current threshold. Finally, the top- k objects in $\mathcal{R}$ are returned.

Early-Stopping Distance Evaluation. Algorithm 7 implements the early-stopping distance evaluation routine. Given a candidate object o , the aggregated distance $d_\alpha(q, o)$ is computed incrementally following the permutation π , where feature indices are sorted in non-increasing order of the query weights.

At each iteration, the partial sum d_o accumulates a weighted per-feature distance term. Since all distance components are non-negative, the partial sum is a lower bound of the final aggregated distance. If the partial sum exceeds the current threshold τ , the computation terminates immediately and returns $+\infty$, indicating that o cannot improve the current beam.

Algorithm 6: Weight-Aware Search on WTNG

Input: Graph G ; query q with weight vector α ; beam width ef_{search} ; entry-point set EP ; return size k ; template set $\mathcal{T}$; number of templates n_t

Output: Top- k approximate nearest neighbors $\mathcal{R}$

```
1 Procedure WTNGSearch( $G, q, EP, k, \mathcal{T}, n_t$ ):
2   Min-heap  $\mathcal{S} \leftarrow \emptyset$ 
3   Max-heap  $\mathcal{R} \leftarrow \emptyset$ 
4   Visited  $\leftarrow \emptyset$ 
5   foreach  $s \in EP$  do
6     Visited.add( $s$ )
7      $d_s \leftarrow d_\alpha(q, s)$ 
8      $\mathcal{S}.add(d_s, s)$ 
9      $\mathcal{R}.add(d_s, s)$ 
10    if  $|\mathcal{R}| > ef_{\text{search}}$  then
11      pop the farthest object from  $\mathcal{R}$ 
12   $\mathcal{T}_q \leftarrow$  the  $n_t$  templates in  $\mathcal{T}$  closest to  $\alpha$ 
13   $\pi \leftarrow (i_1, \dots, i_m)$  s.t.  $\alpha_{i_1} \geq \dots \geq \alpha_{i_m}$ 
14  while  $\mathcal{S}$  is not empty do
15     $(d_v, v) \leftarrow$  extract-min from  $\mathcal{S}$ 
16    if  $|\mathcal{R}| = ef_{\text{search}}$  then
17       $(d_u, u) \leftarrow \mathcal{R}.\text{peek-max}()$ 
18       $\tau \leftarrow d_u$ 
19      if  $d_v > \tau$  then
20        break
21     $NS(v) \leftarrow \{o \in G[v] \mid \exists \alpha^{(j)} \in \mathcal{T}_q \text{ s.t. } b_j(v, o) = 1\}$ 
22    foreach  $o \in NS(v)$  and  $o \notin \text{Visited}$  do
23      Visited.add( $o$ )
24      if  $|\mathcal{R}| < ef_{\text{search}}$  then
25         $d_o \leftarrow d_\alpha(q, o)$ 
26         $\mathcal{S}.add(d_o, o)$ 
27         $\mathcal{R}.add(d_o, o)$ 
28      else
29         $d_o \leftarrow \text{AggDistEarlyStop}(o, q, \alpha, \tau, \pi)$ 
30        if  $d_o < \tau$  then
31           $\mathcal{S}.add(d_o, o)$ 
32           $\mathcal{R}.add(d_o, o)$ 
33          if  $|\mathcal{R}| > ef_{\text{search}}$  then
34            pop the farthest object from  $\mathcal{R}$ 
35  return Top- $k$  elements of  $\mathcal{R}$ 
```

Otherwise, once all features have been processed, the full aggregated distance is returned. This mechanism avoids unnecessary distance computations for candidates that are clearly worse than the current threshold.

G. Dataset Description

A detailed description of our datasets is provided below.

- OpenImage [48], [53]: The OpenImage dataset is an open benchmark connecting vision and language. It comprises

Algorithm 7: Early-Stopping Distance Evaluation

Input: Object o ; query q ; weight vector α ; threshold τ ; permutation π

Output: Aggregated distance d_o (or $+\infty$ if early-stopped)

```
1 Procedure AggDistEarlyStop( $o, q, \alpha, \tau, \pi$ ):
2    $d_o \leftarrow 0$ 
3   for  $t \leftarrow 1$  to  $m$  do
4      $i \leftarrow \pi[t]$ 
5      $d_o \leftarrow d_o + \alpha_i \cdot d(q.e_i, o.e_i)$ 
6     if  $d_o > \tau$  then
7       return  $+\infty$ 
8   return  $d_o$ 
```

around 500K training images and 41K validation images. Each image is paired with localized narratives provided by annotators. To construct the feature representations for each object, images are converted into 768-dimensional vectors using ViT [54], and localized narratives are transformed into 768-dimensional vectors using BERT [55]. We use these two vectors for the two-vector setting. For the three-vector setting, we further translate the localized narratives into Chinese and encode the translated text into an additional 768-dimensional vector using BERT. The query set Q ($|Q| = 1,000$) is randomly selected from the validation set.

- ImageNet [49], [56]: ImageNet is a large-scale image database organized according to the WordNet hierarchy. We adopt three ImageNet-derived embedding sets consisting of 1.28M vectors: ImageNet-DINOv2, ImageNet-EVA02, and ImageNet-ConvNeXt, which are generated using different embedding models as described in [57]. We use the above three vectors for the three-vector setting, whereas for the two-vector setting, we retain only the ImageNet-DINOv2 and ImageNet-EVA02 vectors. The query set Q ($|Q| = 1,000$) is randomly sampled from the entire dataset, and the remaining vectors are used as the database.
- MSMARCO [50]: MSMARCO is a large-scale web search dataset widely used for open-domain retrieval and passage ranking. We use approximately 3.2M documents from the corpus as database objects. To construct a multi-vector representation for each object, we split its textual content into three parts: the title and other short textual fields are encoded as one vector, while the main body content is divided into two segments and encoded as two separate vectors. Each part is encoded into a 768-dimensional embedding using BERT [55], resulting in three 768-dimensional vectors per database object. Queries are encoded by the same encoder to obtain 768-dimensional query embeddings. The query set Q ($|Q| = 1,000$) is randomly sampled from the entire dataset, and the remaining vectors are used as the database.

- CC3M [31], [58]: CC3M is a large-scale image-text dataset containing approximately 3.3M pairs. The captions are collected from webpage alt text and then filtered to remove noisy descriptions. Due to invalid or expired image URLs, we restrict our experiments to the 290K image-text pairs that can be successfully downloaded. Images are converted into 768-dimensional vectors using ViT [54], and captions are transformed into 768-dimensional vectors using BERT [55]. We use these two vectors for the two-vector setting. For the three-vector setting, we further translate the captions into Chinese and encode the translated text into an additional 768-dimensional vector using BERT. The query set Q ($|Q| = 1000$) is randomly selected from the validation set.

H. Two-vector Setting and Four-vector Setting

For the two-vector dataset, we randomly sample two embeddings from the all original three-vector dataset: OpenImage, ImageNet, MSMARCO and CC3M.

For the four-vector dataset, we generate an additional embedding using a different ViT model for OpenImage dataset, where the original ViT model is google/vit-base-patch16-224-in21k and the new ViT model is facebook/dino-vitb16.

For the two-vector setting (i.e., $m = 2$), the simplex constraint $\alpha_1 + \alpha_2 = 1$ admits a one-parameter representation. We let $\alpha = \alpha_1$, so that $\alpha = (\alpha, 1 - \alpha)$ with $\alpha \in [0, 1]$. Following [30], we partition α into five intervals: $[0, 0.2]$, $[0.2, 0.4]$, $[0.4, 0.6]$, $[0.6, 0.8]$, and $[0.8, 1.0]$. For each interval, we construct a query set by sampling one α for each query. In the main text, we report the overall performance over all five intervals, together with the results for three representative intervals: $[0.2, 0.4]$, $[0.4, 0.6]$, and $[0.6, 0.8]$.

For the four-vector setting (i.e., $m = 4$), the simplex constraint is given by $\sum_{i=1}^4 \alpha_i = 1$ with $\alpha_i \geq 0$. We parameterize the weight vector as $\alpha = (\alpha_1, \alpha_2, \alpha_3, 1 - \alpha_1 - \alpha_2 - \alpha_3)$. Following the same evaluation protocol, we partition the weight space into representative regions covering single-modality-dominant, two-modality-dominant, three-modality-balanced, and fully balanced cases. For each region, we construct a query set by sampling one valid α for each query within the corresponding simplex subregion. The specific regions are: $(\alpha_1, \alpha_2, \alpha_3) \in [0.6, 0.8] \times [0, 0.2] \times [0, 0.2]$, $[0, 0.2] \times [0.6, 0.8] \times [0, 0.2]$, $[0, 0.2] \times [0, 0.2] \times [0.6, 0.8]$, $[0, 0.1]^3$, $[0.4, 0.5] \times [0.4, 0.5] \times [0, 0.1]$, $[0.4, 0.5] \times [0, 0.1] \times [0.4, 0.5]$, $[0, 0.1] \times [0.4, 0.5] \times [0.4, 0.5]$, $[0.3, 0.4]^3$, $[0.4, 0.5] \times [0.1, 0.2]^2$, $[0.1, 0.2] \times [0.4, 0.5] \times [0.1, 0.2]$, $[0.1, 0.2]^2 \times [0.4, 0.5]$, $[0.2, 0.3]^3$.

I. Three-Vector MVQ Performance

The complete experimental results for the three-vector MVQ setting are shown in Figure 12, corroborating the observations reported in the main text.

J. Two-Vector MVQ Performance

The complete experimental results for the two-vector MVQ setting are shown in Figure 13, corroborating the observations reported in the main text.

K. Four-Vector MVQ Performance

The complete experimental results for the four-vector MVQ setting are shown in Figure 14, corroborating the observations reported in the main text.

L. Full Results of Ablation Study and Sensitivity Analysis

The complete results of the ablation studies on the WTNG edge pruning strategy and edge activation mechanism, as well as the sensitivity analysis of M and $ef_{\text{construction}}$, are reported in Figure 15, Figure 16, and Figure 17, respectively, corroborating the observations reported in the main text.

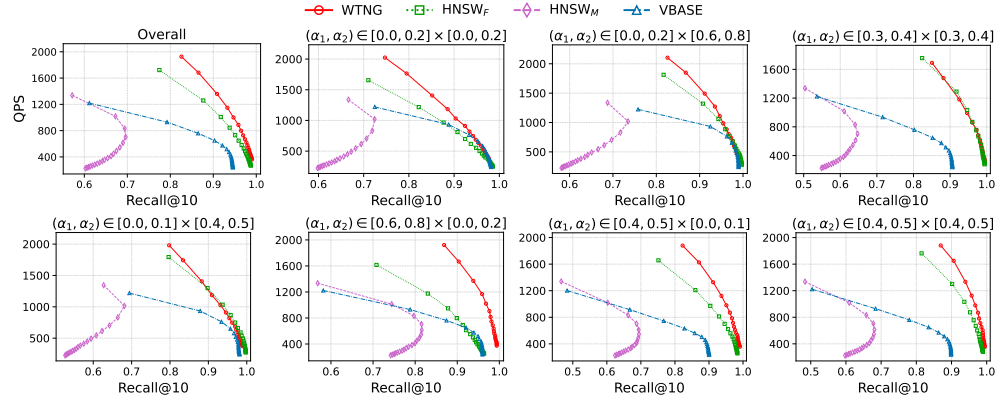
M. Additional Ablation Study and Sensitivity Analysis

Ablation Study of Candidate Acquisition Method. Recall that in Section III-B, the K -skyband candidates are approximated via skyline computation. Skyline processing can be carried out either approximately using subspace skyline methods (as adopted by our WTNG), or exactly using classical skyline algorithms. Classical exact methods admit time complexity $O(N \log^{m-2} N)$ [59], where N is the number of data points and $m \geq 3$.

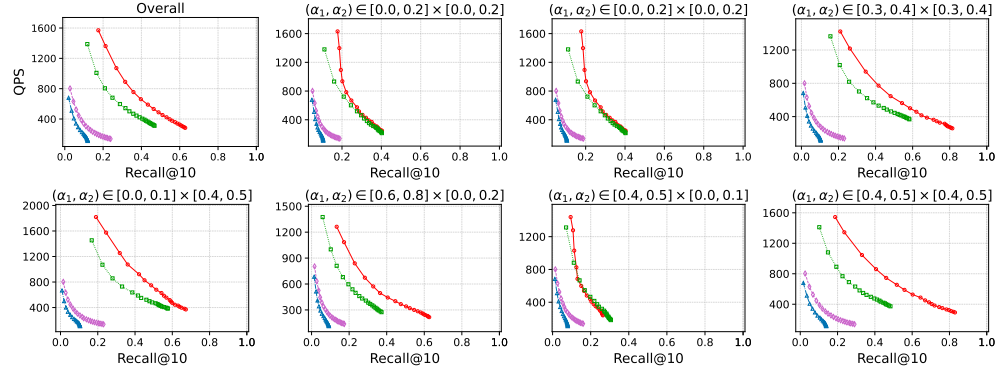
One representative approach is the sort-and-scan method [43], implemented using a binary indexed tree (BIT) [60]. The method begins by sorting all points according to one feature and then scans them in that order. During the scan, a binary indexed tree maintains a compact summary indexed by a second objective, allowing each new point to be tested for dominance via a logarithmic-time prefix query instead of comparing against all prior points.

We replace the subspace skyline module in WTNG with the sort-and-scan method (implemented via binary index tree), denoted as WTNG+BIT. We compare WTNG and WTNG+BIT on the openimage dataset, with results reported in Figure 18. In general, the two variants achieve comparable performance. However, under certain weight configurations, their performance can differ noticeably, suggesting that the skyline module affects the candidate composition. Compared with the BIT-based exact skyline computation, our approximate subspace skyline may admit additional vertices beyond the exact solution set, increasing candidate diversity and thereby altering the search trajectory for those α , while keeping the overall performance broadly similar.

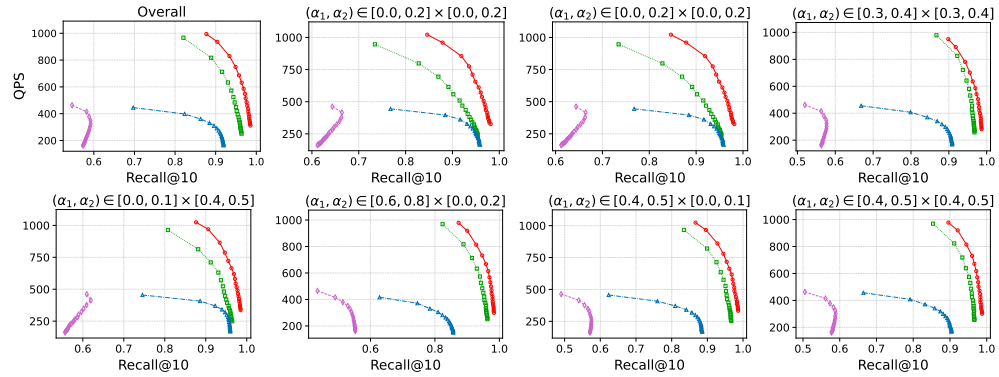
Ablation Study of Early-Stopping Mechanism. To evaluate the effectiveness of the early-stopping mechanism, in which partial distance accumulation is compared against full distances to terminate computation early, we construct a variant of WTNG without this component, denoted as WTNG_{w/o stop}. Experimental results on the openimage dataset are shown in Figure 19. Overall, WTNG achieves approximately a $1.2 \times$ speedup over WTNG_{w/o stop} at 90% recall. This suggests that there often exists a non-trivial gap in the query-to-candidate



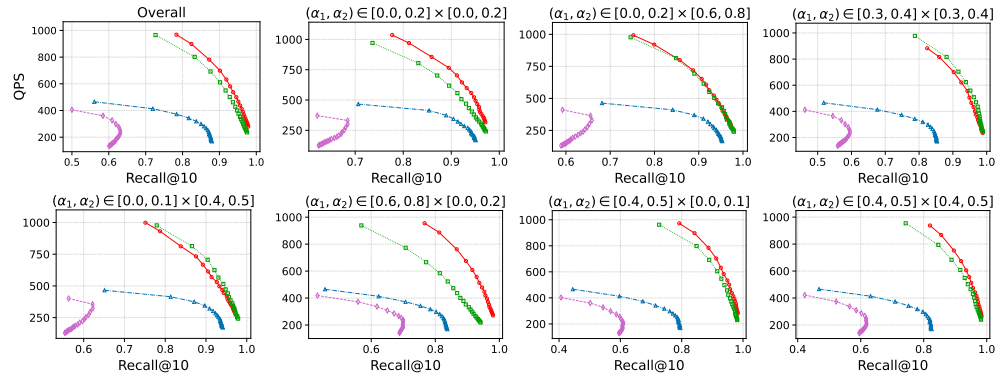
(a) OpenImage



(b) ImageNet



(c) MSMARCO



(d) CC3M

Fig. 12: Accuracy-Efficiency Tradeoff Results in Three-Vector Setting (Upper and Right is Better)

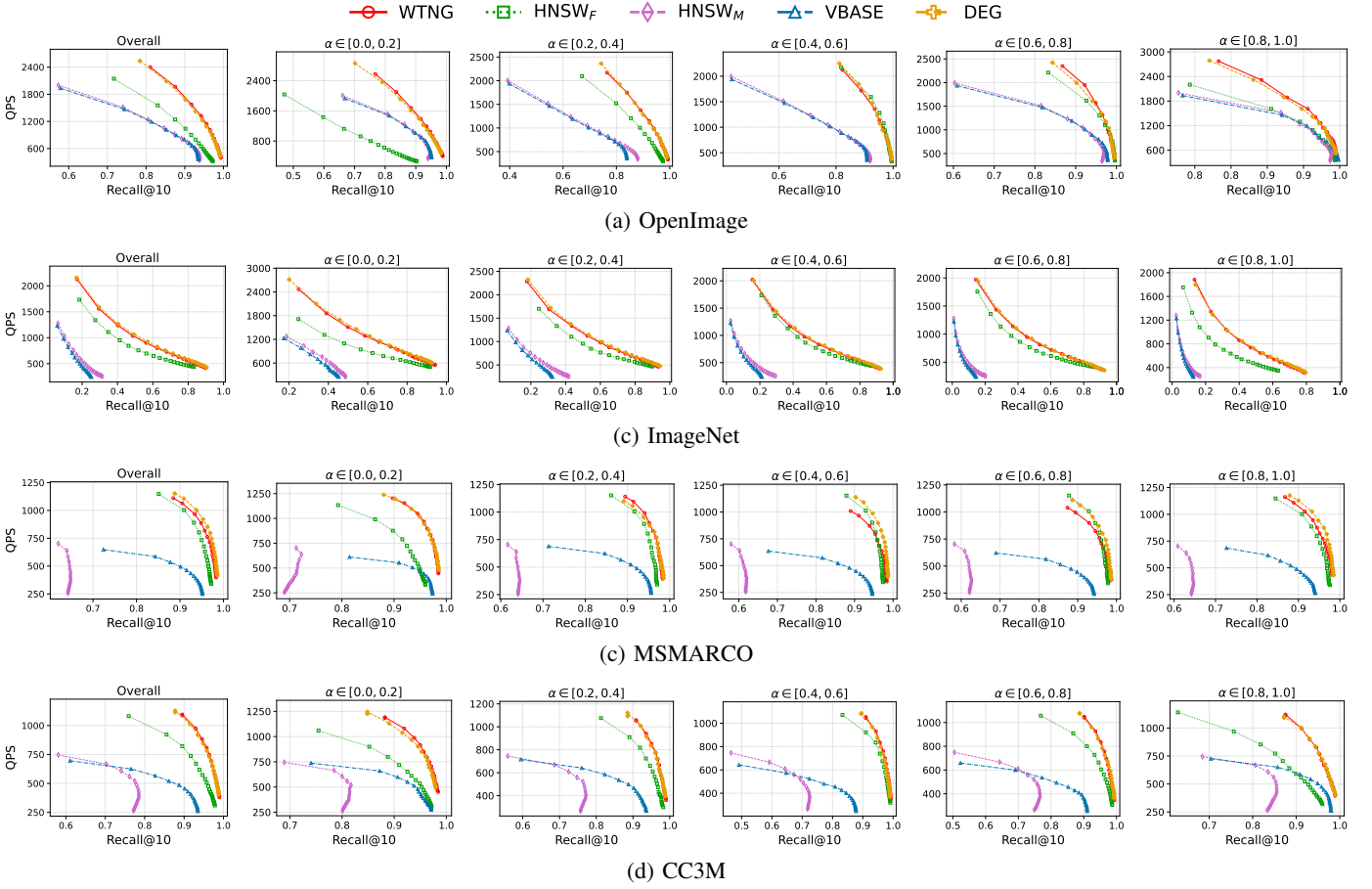


Fig. 13: Accuracy-Efficiency Tradeoff Results in Two-Vector Setting (Upper and Right is Better)

distances, separating strong candidates from inferior ones. As a result, partial distance accumulation frequently exceeds the current best value, enabling early termination of unnecessary computations. Although $\text{WTNG}_{\text{w/o stop}}$ may perform comparably to WTNG in some cases, WTNG consistently dominates across all settings, with its QPS–Recall@10 curve lying above that of $\text{WTNG}_{\text{w/o stop}}$. These results confirm the effectiveness of the early-stopping mechanism in improving search efficiency.

Sensitivity to the Size of Entry Points. Figure 20 presents the sensitivity of WTNG to the size bound B of the entry-point set. The entry-point set determines the number of initial starting vertices used during search, which directly affects the exploration breadth and search overhead. We choose $B = 12, 15, 18$ and refer to them as WTNG_{12} , WTNG_{15} and WTNG_{18} . We can observe that WTNG_{12} exhibits noticeably worse performance compared to WTNG_{15} and WTNG_{18} , indicating that an insufficient number of entry points limits search coverage and degrades accuracy-efficiency tradeoffs. In contrast, raising the entry-point-set size bound beyond 15 yields only marginal improvements, as WTNG_{15} and WTNG_{18} achieve nearly identical QPS–Recall@10 curves across all reported settings. These results indicate that a moderate entry-point-set size budget (i.e., 15) strikes a favorable balance between

exploration diversity and search efficiency.

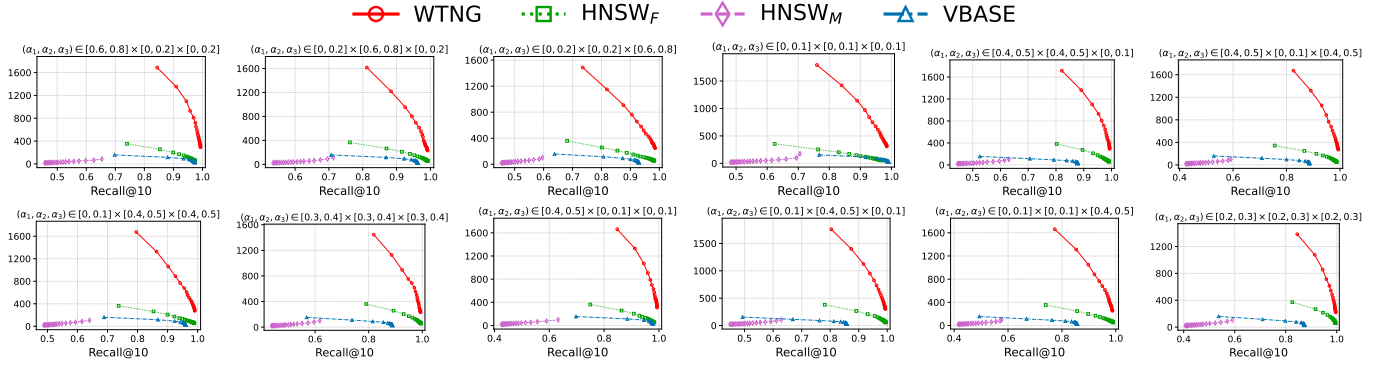


Fig. 14: Accuracy-Efficiency Tradeoff Results in Four-Vector Setting on OpenImage (Upper and Right is Better)

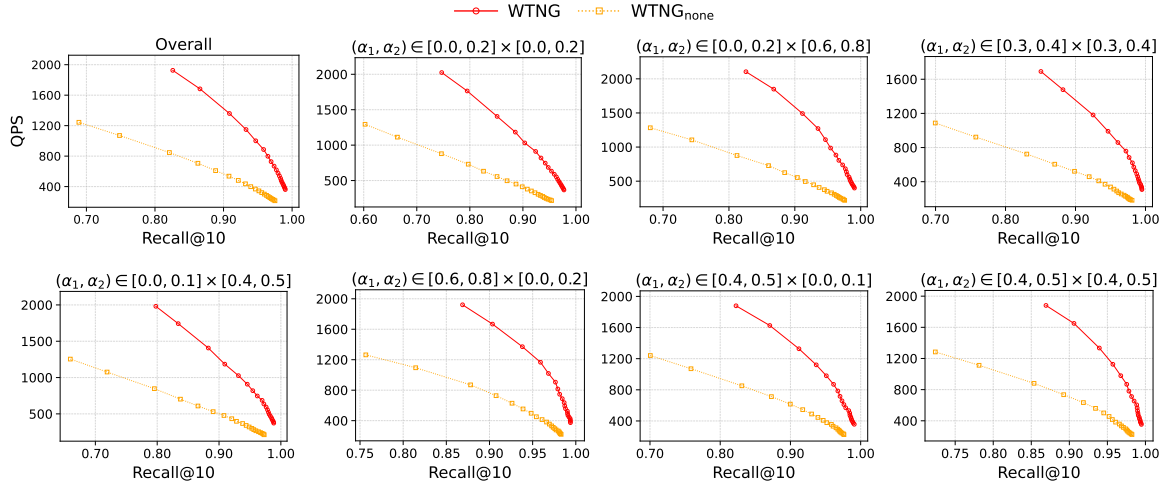


Fig. 15: Ablation Study for Edge Prune Strategy

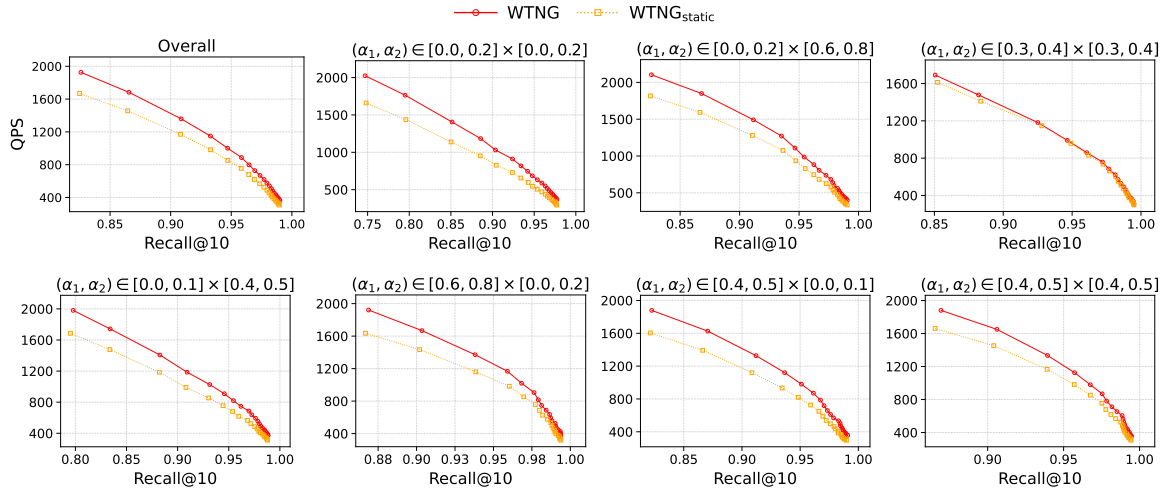


Fig. 16: Ablation Study for Edge Activation Mechanism

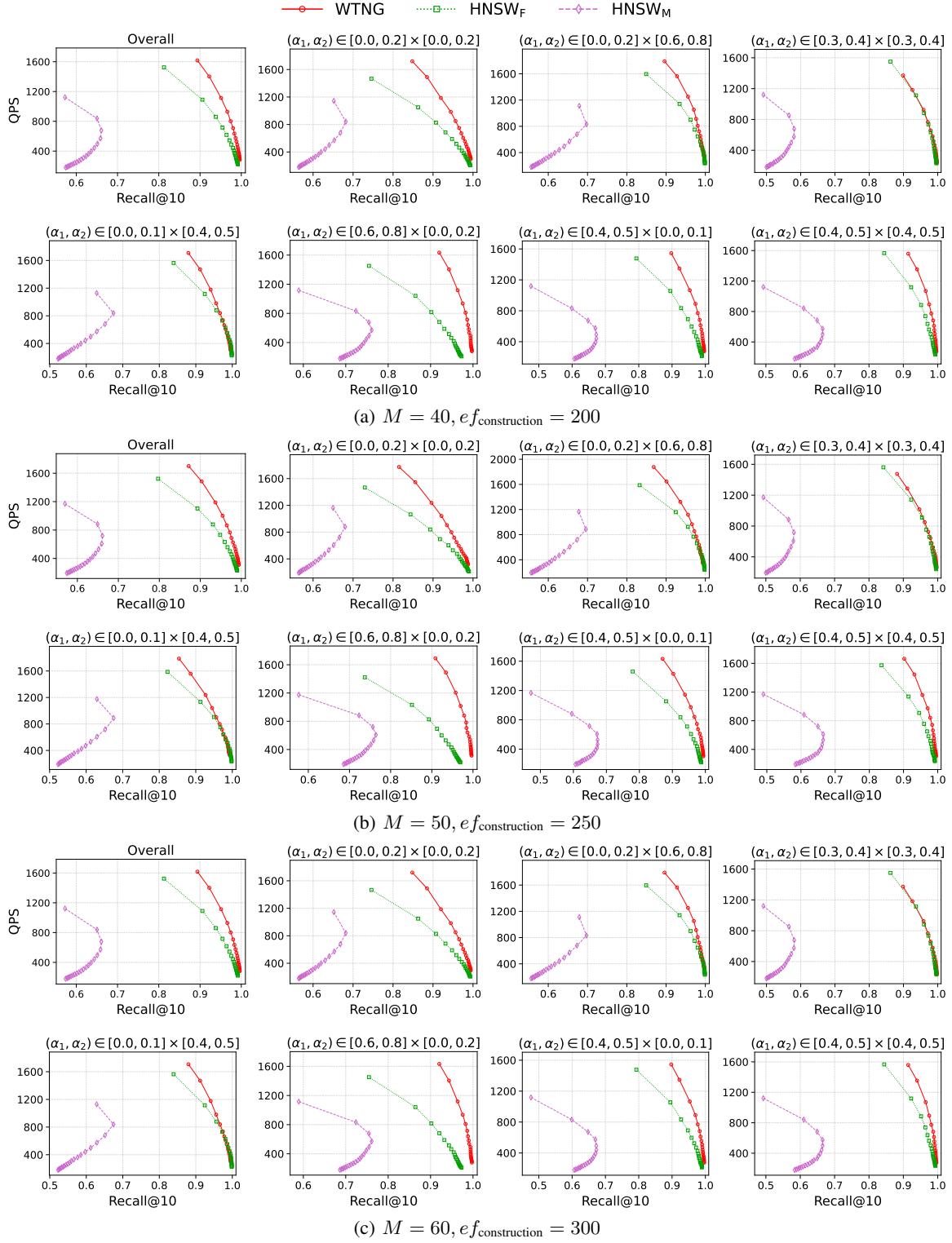


Fig. 17: Performance with Varying M and $ef_{\text{construction}}$

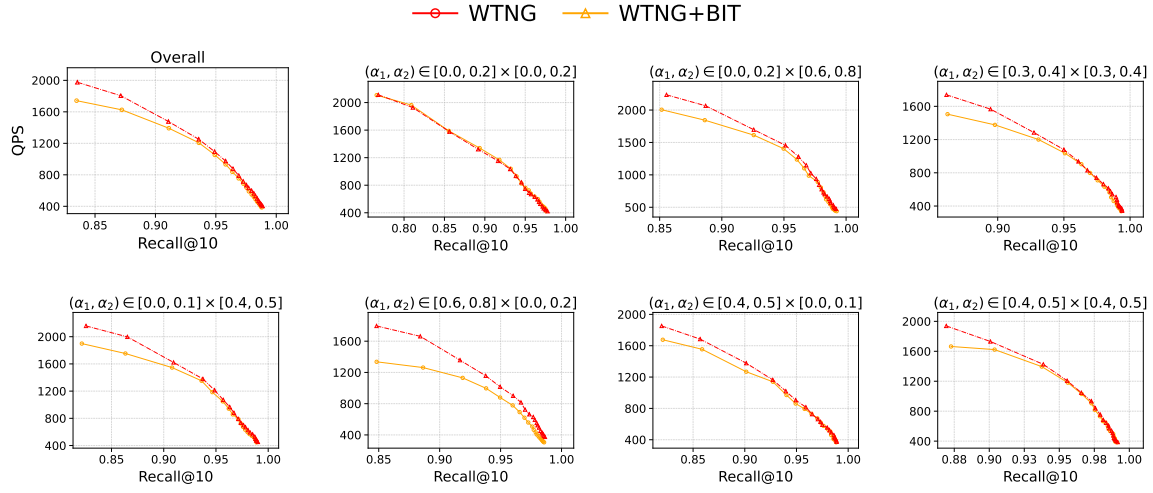


Fig. 18: Ablation Study for Candidate Acquisition Method

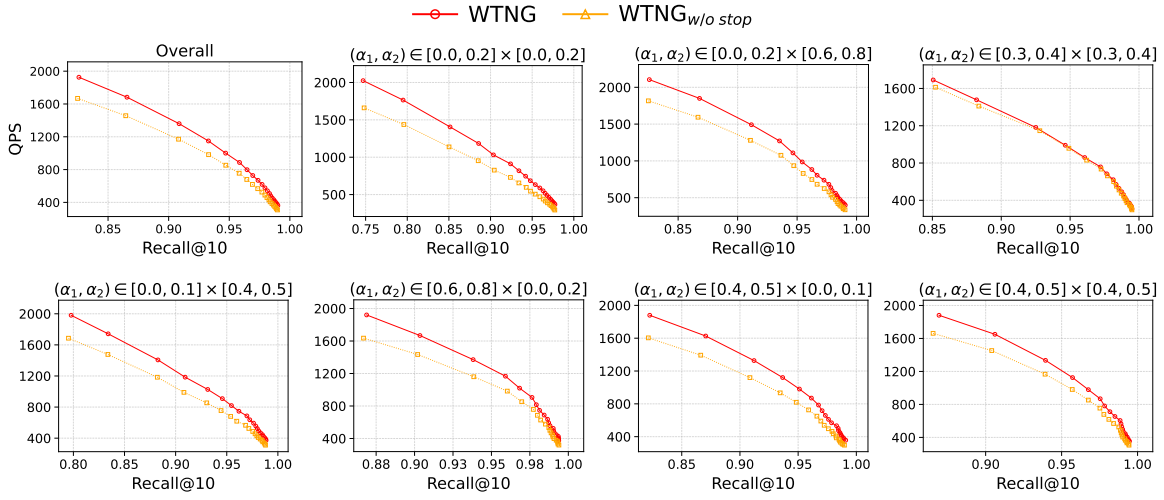


Fig. 19: Ablation Study for Early-Stopping Mechanism

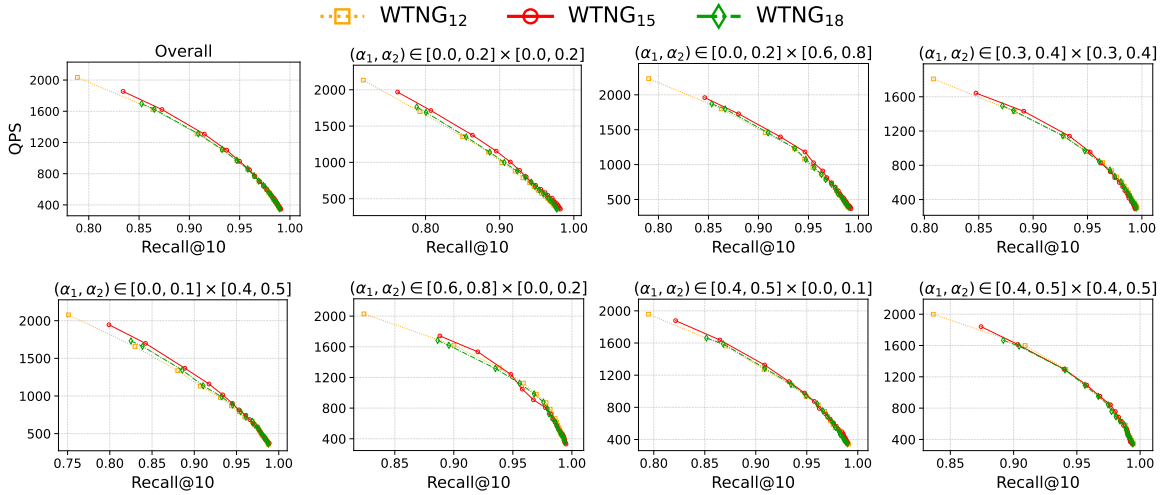


Fig. 20: Sensitivity Study of Entry Point Size